

imperative commands vs **declarative manifests** in Kubernetes in **deep detail**, breaking down what each method is, how they work, their pros and cons, and when to use which.

What Are They?

1. Imperative Commands

You tell Kubernetes what to do step-by-step using the **kubectl** command-line tool.

- Think of it like giving **direct instructions**: "Create a pod with this image", "Delete that deployment", "Scale this to 5 replicas".
- You do **not** maintain a file or configuration to describe the system — you're directly issuing commands.

Example:

```
kubectl run nginx --image=nginx
```

This creates a pod running an **nginx** container.

2. Declarative Manifests

You **define the desired state** of the system in a configuration file (usually YAML or JSON), and tell Kubernetes: "Make the system look like this."

- You write manifests to describe what the system should look like — such as what pods, services, deployments should exist.
- Kubernetes reconciles the current state to match your declared desired state.

- This is central to Infrastructure as Code (IaC) and **GitOps** practices.

 Example Manifest (nginx-deployment.yaml):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

Apply it using:

```
kubectl apply -f nginx-deployment.yaml
```

 Key Differences

Feature	Imperative	Declarative
Usage Style	Procedural — do this now	Descriptive — here's what the system should look like
Tool	<code>kubectl create</code> , <code>kubectl delete</code> , <code>kubectl run</code> , etc.	<code>kubectl apply -f</code> , <code>kubectl diff</code> , etc.

Reproducibility	Not reproducible unless commands are scripted	Fully reproducible if files are kept in version control
Version Control	Hard to track changes	Easy to track (YAML files can be stored in Git)
Team Collaboration	Manual sharing of commands	Teams can collaborate on versioned manifests
State Management	You are responsible for tracking state	Kubernetes and controllers manage state automatically
Risk of Drift	High — someone might change something manually	Low — desired state is always re-applied
Suitability	Best for learning, debugging, ad-hoc tasks	Best for production, automation, CI/CD pipelines

Conceptual Analogy

- **Imperative:** Like baking a cake with a recipe you *perform step-by-step* (mix flour, add eggs, bake at 350°F).
 - **Declarative:** Like saying “I want a chocolate cake” and letting a chef or a system make it based on the ingredients and methods it knows.
-

When to Use What?

Scenario	Use Imperative	Use Declarative
Quick test or learning	✓ Yes	✗ Not necessary
One-time script or automation	✓ Yes	✓ Possible
Production environments	✗ Risky	✓ Best Practice
CI/CD pipelines (GitOps)	✗ Not scalable	✓ Essential

Configuration tracking in Git	 Difficult	 Easy
Collaboration among team members	 Poor	 Excellent

Hybrid Approach (Advanced)

You can use both together! For example:

- Start with imperative commands to prototype.

Export objects using:

```
kubectl get deployment nginx -o yaml > nginx.yaml
```

- Then switch to **declarative** management from there.
-

Summary

Aspect	Imperative	Declarative
Fast and simple		
Reusable and version-controlled		
Scales to teams and environments		
Best for CI/CD and GitOps		
Command-driven		
File-driven (YAML/JSON)		

If you're working on a **serious or long-term project**, **declarative manifests** are the way to go. They align with modern DevOps practices and make your infrastructure predictable, manageable, and auditable.

#Imperative Commands on K8s

Sure! Below are **imperative kubectl** commands to:

1. Run a **Pod**
2. Create a **Deployment** with replicas
3. **Expose** a Deployment as a Service

✓ 1. Run a Pod

This creates a single pod named **my-nginx** using the **nginx** image:

```
kubectl run my-nginx --image=nginx
```

This is a **bare pod**, not recommended for production because it's not managed (e.g. auto-restarted if it fails).

✓ 2. Create a Deployment with Replicas

This creates a Deployment named **nginx-deployment** with 3 replicas:

```
kubectl create deployment nginx-deployment --image=nginx  
kubectl scale deployment nginx-deployment --replicas=3
```

Alternatively, you can combine it like this (if supported by your version of `kubectl`):

```
kubectl create deployment nginx-deployment --image=nginx --replicas=3
```

Note: The `--replicas` flag is not supported in older versions during `kubectl create`, hence `kubectl scale` is often used as a second step.

✓ 3. Expose a Deployment

This exposes the `nginx-deployment` as a Service of type `ClusterIP` (default) on port 80:

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80
```

To expose it externally via a **LoadBalancer** (for example, in cloud environments):

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80  
--type=LoadBalancer
```

For a **NodePort** (for local access in Minikube or bare metal clusters):

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80 --type=NodePort
```

🎯 Summary

Task	Imperative Command
Run a Pod	<code>kubectl run my-nginx --image=nginx</code>
Create a Deployment	<code>kubectl create deployment nginx-deployment --image=nginx</code>
Scale Deployment	<code>kubectl scale deployment nginx-deployment --replicas=3</code>
Expose Deployment (ClusterIP)	<code>kubectl expose deployment nginx-deployment --port=80 --target-port=80</code>

Expose Deployment
(NodePort)

```
kubectl expose deployment nginx-deployment --port=80  
--target-port=80 --type=NodePort
```

Expose Deployment
(LoadBalancer)

```
kubectl expose deployment nginx-deployment --port=80  
--target-port=80 --type=LoadBalancer
```

#K8s Declarative Manifest

- Create pod.yml
- vi pod.yml

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: my-nginx  
  labels:  
    app: nginx  
spec:  
  containers:  
    - name: nginx  
      image: nginx:latest  
      ports:  
        - containerPort: 80
```

#to run this file

- kubectl apply -f pod.yml

#Create a Deployment file

- vi deploy.yml

```
apiVersion: apps/v1  
kind: Deployment  
metadata:
```

```
name: nginx-deployment
labels:
  app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

#Apply the deployment

```
- kubectl apply -f deploy.yml
```

#This will run the pods with deployments and replicas.

#Create a service.yml

vi service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
spec:
  type: LoadBalancer
  selector:
```

app: nginx

ports:

- protocol: TCP

port: 80

targetPort: 80